

# Costanti in Java

In Java, le costanti sono valori che, una volta assegnati, non possono essere modificati durante l'esecuzione del programma. Sono utili per rappresentare dati fissi come valori matematici (es.  $\Pi$ ), parametri di configurazione o valori che non devono cambiare per evitare errori logici.

## Dichiarazione con final

Per definire una costante in Java si utilizza la parola chiave **final** prima del tipo di dato. Una volta inizializzata, qualsiasi tentativo di riassegnazione genera un errore in compilazione.

```
final double PI = 3.14159;
final int MESI_NELL_ANNO = 12;

// Tentativo di modifica (genera errore):
// PI = 3.14; // Errore: cannot assign a value to final variable PI
```

## Convenzioni di nomenclatura

Per distinguere chiaramente le costanti dalle variabili, si usa la convenzione di scriverne il nome in maiuscolo, con le parole separate da underscore (\_):

```
final String NOME_APPLICAZIONE = "GestioneClienti";
final int LIMITE_TENTATIVI = 3;
```

## Tipi di costanti

Come le variabili, le costanti in Java possono essere classificate in base al loro contesto:

- Costanti *locali*: dichiarate all'interno di un metodo.
- Costanti di *istanza*: campi final non static, inizializzati nel costruttore.
- Costanti di *classe*: dichiarate come public static final, condivise tra tutte le istanze.

Dipendiamo questa lista.

### Costanti locali

Le costanti locali sono variabili dichiarate con final all'interno di un metodo, blocco o costrutto (come un ciclo).

#### Dichiarazione e uso

```
public void calcolaArea() {
    final double PI = 3.14159;
    final int raggio = 5;
    double area = PI * raggio * raggio;
    // PI = 3.14; // Errore: non puoi riassegnare una costante
}
```

## Caratteristiche principali

*Visibilità limitata*: esistono solo all'interno del blocco in cui sono dichiarate.

### Usi comuni

Parametri di metodo:

```
public void elabora (final String input) {
    // input = "nuovo"; // Errore
}
```

Variabili in cicli foreach:

```
for (final String item : lista) {
    IO.println (item);
}
```

Supporto a lambda e classi anonime: (spiegato più avanti) le variabili locali catturate devono essere final o effectively final (non riassegnate).

## Costanti di istanza

Le costanti di istanza sono variabili dichiarate con `final` all'interno di una classe, ma senza la parola chiave `static`. Questo significa che ogni oggetto (istanza) della classe ha la sua copia della costante.

### Caratteristiche principali

Appartengono a ogni singola istanza: a differenza delle costanti di classe (`static final`), ogni oggetto ha il proprio valore.

Inizializzazione nel costruttore: possono essere inizializzate solo al momento della dichiarazione o all'interno di un costruttore.

Esempio:

```
public class Cerchio {
    private final double raggio;

    // questo qui sotto è un costruttore (sarà spiegato più avanti)
    public Cerchio (double raggio) {
        this.raggio = raggio; // Inizializzazione valida nel costruttore
    } // this è un riferimento a se stesso nella classe
    // è usato all'interno dei metodi di una classe
    // in questo caso per distinguere raggio-istanza da raggio-parametro
    public double getRaggio() {
        return raggio;
    }
}
```

Ogni istanza di `Cerchio` ha un proprio raggio fisso:

```
Cerchio c1 = new Cerchio(5.0);
Cerchio c2 = new Cerchio(10.0);
// c1.raggio e c2.raggio sono costanti diverse
```

### Regole importanti

- Non possono essere assegnate in metodi normali (solo in costruttori).
- Devono essere inizializzate in tutti i costruttori, altrimenti il codice non compila.
- Se una classe ha più costruttori, tutti devono inizializzare la costante di istanza, oppure usare `this()` per delegare a un costruttore che la inizializza.

## Costanti di classe in Java

Le costanti di classe sono variabili dichiarate con `public static final` all'interno di una classe. Sono condivise tra tutte le istanze della classe.

Esempio di dichiarazione:

```
public class Costanti {
    public static final double PI = 3.14159;
    public static final int MAX_UTENTI = 100;
    public static final String VERSIONE = "1.0.0";
}
```

# Costrutti di programmazione

In ogni linguaggio di programmazione esistono costrutti che permettono al programmatore di controllare la sequenza di esecuzione delle istruzioni. Possiamo dividere questi costrutti in due categorie principali:

## Costrutti condizionali

- *if - else*: esegue blocchi di codice in base a una condizione booleana.
- *switch*: seleziona un caso tra più opzioni, utile con valori interi, stringhe o enum.

## Costrutti iterativi (cicli)

- *for*: per iterazioni con contatore.
- *while*: ripete finché la condizione è vera.
- *do - while*: come while, ma esegue almeno una volta.
- *for - each*: Ottimizzato per scorrere elementi di array o collezioni

## if - else

### Sintassi base

```
if (condizione) {  
    // istruzioni se vera  
} else {  
    // istruzioni se falsa  
}
```

Se la condizione if deve eseguire solo una istruzione, allora le parentesi graffe posso essere omesse. Stesso per else. Ma è consigliato metterle.

```
if (condizione)  
    Istruzione;  
else  
    AltraIstruzione;
```

La condizione deve restituire un valore booleano (true o false).

Esempio semplice:

```
int numero = 10;  
if (numero > 0) {  
    IO.println("Positivo");  
} else {  
    IO.println("Non positivo");  
}
```

Diagramma if

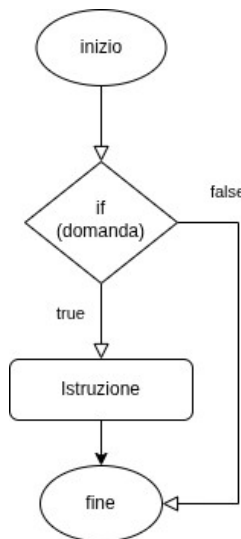
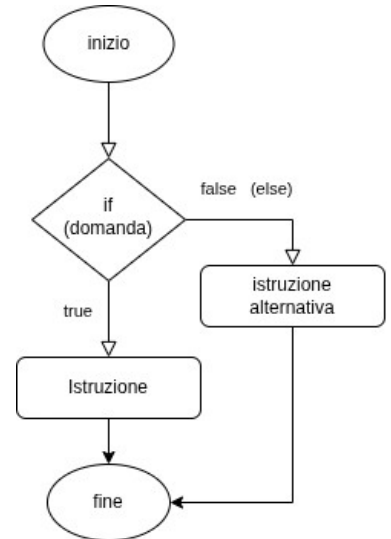


Diagramma if - else



### if - else a catena

```
int voto = 7;  
if (voto < 6) {  
    IO.println("Insufficiente");  
} else if (voto < 8) {  
    IO.println("Buono");  
} else {  
    IO.println("Ottimo");  
}
```

## Errori comuni

- Dimenticare le parentesi graffe

```
if (numero > 0)  
    IO.println("Positivo");  
    IO.println("Sempre eseguito"); // Errore logico!
```

**Soluzione:** usare sempre le graffe (anche per una singola istruzione).

- Usare = invece di ==

```
if (numero = 5) // Errore: assegnazione, non confronto
```

- Annidamento errato (dangling else)

```
if (x >= 0)
  if (x <= 10)
    IO.println("Tra 0 e 10");
  else
    IO.println("Negativo"); // Si riferisce al secondo if!
```

**Soluzione:** usare le graffe per chiarire il blocco:

```
if (x >= 0) {
  if (x <= 10) {
    IO.println("Tra 0 e 10");
  }
} else {
  IO.println("Negativo");
}
```

## Ricordiamo l'esistenza dell'operatore ternario ?:

L'operatore ternario in Java è un operatore condizionale che permette di scrivere in modo conciso una semplice struttura if-else. È l'unico operatore in Java che richiede tre operandi, da cui il nome ternario.

### Sintassi

```
variabile = (condizione) ? valoreSeVero : valoreSeFalso;
```

Se la condizione è vera, viene assegnato valoreSeVero, altrimenti valoreSeFalso.

### Esempio pratico

```
int eta = 18;
String stato = (eta >= 18) ? "Maggiorenne" : "Minorenne";
IO.println(stato); // Output: Maggiorenne
```

Equivalenza

```
String stato;
if (eta >= 18) {
  stato = "Maggiorenne";
} else {
  stato = "Minorenne";
}
```

### Errori comuni

- Tipi incompatibili:

Entrambe le espressioni devono restituire lo stesso tipo (o un tipo comune).

```
// Errore di compilazione
String risultato = (true) ? "OK" : 0; // String vs int
```

Una soluzione a questo esempio specifico sarebbe quella di usare la classe superiore a tutte, Object:

```
Object risultato = (true) ? "OK" : 0; // Entrambi sono Object
```

e poi far decidere in seguito al programma cosa fare, ma il rischio è quello di complicare ciò che in partenza partiva come una semplificazione.

Quindi la regola principale rimane quella di restituire espressioni dello stesso tipo, così da non avere problemi di sorta.

- Uso con valori null:  
Chiamare metodi su null causa NullPointerException.

```
String input = null;
// Causa eccezione
String result = input.equals("OK") ? "Successo" : "Fallimento";
```

Esempio **corretto**:

```
String result = "OK".equals(input) ? "Successo" : "Fallimento";
```

- Precedenza degli operatori:  
L'operatore ternario ha bassa precedenza. Usare le parentesi per evitare errori.

```
int a = 10, b = 20;
// Non fa quello che ci si aspetta
IO.println("Risultato: " + a > b ? "A" : "B");
```

Genera "error: bad operand types for binary operator '>'" perché la a (=10) viene concatenata a "Risultato: " e pertanto viene poi richiesto di confrontare una stringa con un numero.

Esempio **corretto**:

```
int a = 10, b = 20;
// Non fa quello che ci si aspetta
IO.println("Risultato: " + ((a > b ? "A" : "B"));
```

- Annidamento eccessivo:  
Rende il codice illeggibile.

```
// Difficile da capire
String categoria = (punteggio > 90) ? "A" : (punteggio > 80) ? "B" : (punteggio > 70) ? "C" : "D";
```

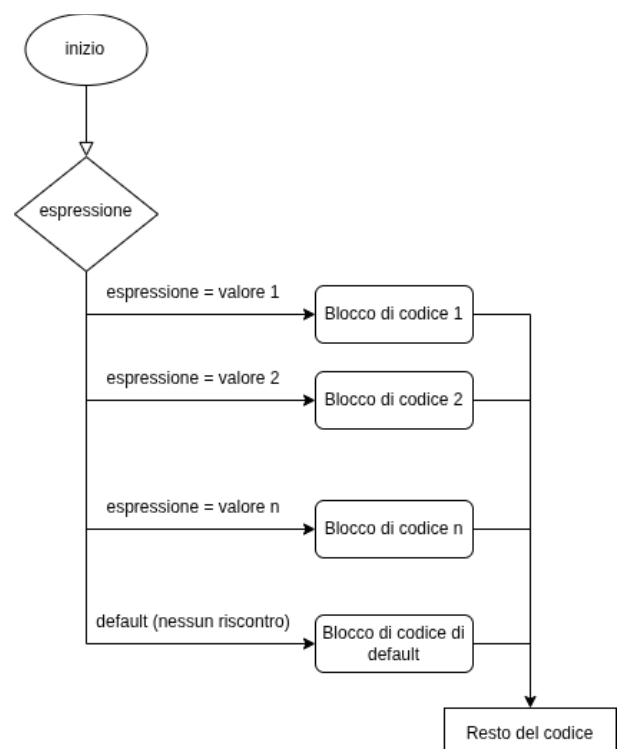
Fortunatamente non siamo dentro ad un foglio di lavoro come excel o google sheet. Qui fortunatamente abbiamo il diritto (e il dovere) di semplificarci la vita. Preferire if-else o switch per logiche complesse.

## switch

Il costrutto switch in Java permette di eseguire diversi blocchi di codice in base al valore di una variabile. È un'alternativa più leggibile a catene ripetute di if-else, specialmente quando si confrontano valori costanti.

### Sintassi classica (con : e break)

```
int giorno = 3;
switch (giorno) {
    case 1:
        IO.println("Lunedì");
        break;
    case 2:
        IO.println("Martedì");
        break;
    case 3:
        IO.println("Mercoledì");
        break;
    default:
        IO.println("Weekend o altro");
}
```



## Sintassi moderna (da Java 14+, con ->)

```
String stagione = "Estate";
switch (stagione) {
    case "Primavera", "Estate" -> IO.println("Stagione calda");
    case "Autunno", "Inverno" -> IO.println("Stagione fredda");
    default -> IO.println("Stagione sconosciuta");
} // da notare l'assenza del break
```

### Errori comuni

- Dimenticare break (fall-through)  
Senza break, il codice continua nei casi successivi.

```
switch (giorno) {
    case 1:
        System.out.println("Lunedì");
    case 2: // Esegue anche questo!
        System.out.println("Martedì");
        break;
}
```

Questo di per sé non è propriamente un errore perché può capitare che ci sia bisogno di raggruppare una stessa risposta per più risultati.

Come ad esempio qui a destra:   
Dalla versione 14 di Java, quando si usa la freccia (->) invece dei due punti (:), non è necessario usare break perché la freccia impedisce automaticamente il fall-through, identificando ogni caso come autonomo.

```
int giorni, mese = 4; // Se non si inizializza giorni
switch (mese) {       // bisogna inizializzarla
    case 1 :           // in ogni caso, anche default.
    case 3 :           // Altrimenti in compilatore
    case 5 :           // restituisce un errore
    case 7 :           // perché nulla garantisce
    case 8 :           // che mese abbia un valore
    case 10 :          // compreso tra 1 e 12,
    case 12 :          // nemmeno se glielo diamo
        giorni = 31;   // fissato, come in questo caso.
        break;
    case 2 :
        giorni = 28;
        break;
    case 4 :
    case 6 :
    case 9 :
    case 11 :
        giorni = 30;
        break;
    default :
        giorni = 0;    // c'è bisogno di questa riga
        break;
}
IO.println ("Il mese " + mese + " ha " +
            giorni + " giorni"); // 4 → 30
```

- Usare tipi non supportati:  
switch **accetta**: **byte**, **short**, **int**, **char**, **String**, **enum** (da Java 7)  
e **var** (in contesti limitati ai tipi di appena citati).  
Non funziona con long, float, double, boolean.
- default non in fondo:  
Anche se tecnicamente permesso, mettere default all'inizio o in mezzo rende il codice confuso.

- Dichiarare variabili senza blocco {}

```
case 1 :
    int x = 10; // Errore: variabile fuori scope
    break;
case 2 :
    int x = 20; // Errore: duplicata
```

Esempio  
corretto:



```
case 1 : {
    int x = 10;
    IO.println(x);
    break;
} // x nasce vive e muore all'interno
// delle parentesi graffe e non si può
// pensare che possa uscire dallo switch
```

## while

Il ciclo while in Java esegue un blocco di codice finché una condizione è vera. La condizione viene valutata all'inizio, quindi il ciclo potrebbe non eseguirsi mai se la condizione è falsa da subito.

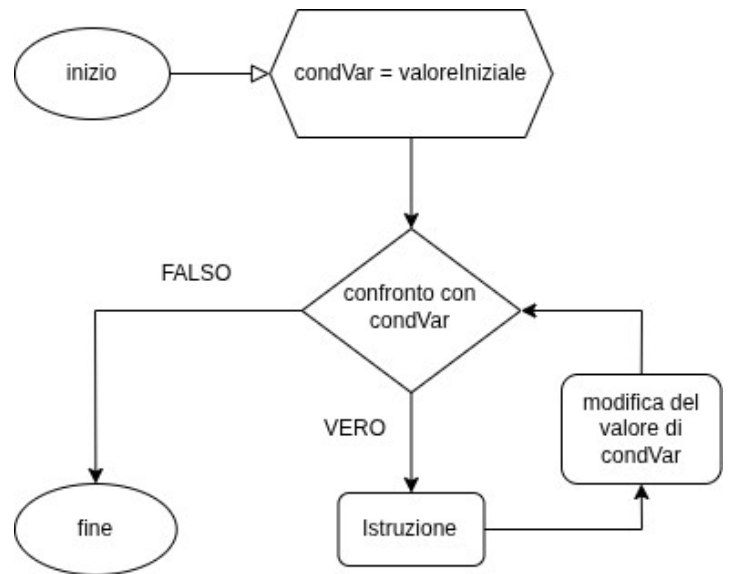
### Sintassi

```
while (condizione) {  
    // istruzioni  
}
```

### Esempio pratico:

```
int i = 1;  
while (i <= 5) {  
    IO.println("Iterazione: " + i);  
    i++;  
} // stampa da 1 a 5
```

se avessimo posto `i++` prima della stampa, i numeri stampati sarebbero stati compresi tra 2 e 6.



### Nota bene

Questo è solo un esempio generico e non categorico al 100%. La modifica della variabile di controllo può avvenire in qualsiasi punto del blocco di codice all'interno del ciclo while, anche subito dopo aver effettuato il confronto.

### Casi d'errore comuni

- Ciclo infinito

Dimenticare di aggiornare la variabile usata nella condizione:

```
int i = 0;  
while (i < 5) {  
    IO.println(i);  
} // Manca i++ → ciclo infinito
```

- Condizione mai soddisfatta:

```
int i = 10;  
while (i < 5) { // Mai vera  
    IO.println(i);  
    i++;  
} // il ciclo non è stato eseguito
```

- Omissione delle parentesi graffe quando ci sono più istruzioni:

Come con `if - else`, se il ciclo è fatto di una sola istruzione, le parentesi graffe non servono. Ma, poiché già la modifica della variabile di controllo è di per sé un'istruzione, vien da sé che ci troviamo obbligati ad usare le parentesi quasi sempre. Quindi sarebbe meglio usarle sempre.

Esempio Corretto:

```
int i = 4;  
while (i < 5) {  
    IO.println(i);  
    i++;  
}
```

Esempio Sbagliato:

```
int i = 4;  
while (i < 5)  
    IO.println(i); // ciclo infinito  
    i++; // Eseguita sempre, fuori dal ciclo
```

## do - while

Il costrutto do-while in Java è un ciclo che esegue il blocco di codice almeno una volta, prima di verificare la condizione. È un ciclo con controllo in coda (exit-controlled loop).

### Sintassi

```
do {  
    // istruzioni  
} while (condizione);
```

### Esempio pratico

```
int i = 10;  
do {  
    IO.println("Valore: " + i);  
    i++;  
} while (i < 5);
```

Anche se la condizione  $i < 5$  è falsa fin dall'inizio, il blocco viene eseguito una volta.

### Confronto con while

|                   | while                         | do - while                  |
|-------------------|-------------------------------|-----------------------------|
| Controllo         | All'Inizio (entry-controlled) | Alla Fine (exit-controlled) |
| Esecuzione minima | 0 volte                       | 1 volta                     |
| Sintassi          | while (cond) { ... }          | do { ... } while (cond);    |
| Punto e virgola   | Non serve                     | Obbligatorio dopo while     |

### Errori comuni

- Dimenticare il punto e virgola:

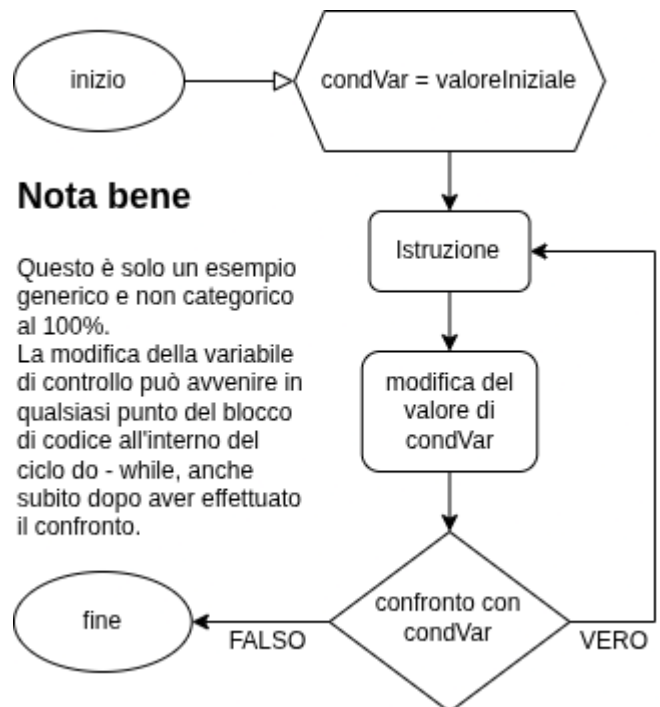
```
do {  
    IO.println("Ciao");  
} while (false) // Errore: manca ;
```

- Ciclo infinito se la condizione non cambia:

```
int x = 1;  
do {  
    IO.println(x);  
} while (x > 0); // x non cambia → ciclo infinito
```

- Usare do-while quando non serve:

Se il ciclo potrebbe non dover mai eseguire, preferire while.





## for

Il costrutto for in Java è un ciclo usato per ripetere un blocco di codice un numero specifico di volte. È particolarmente utile quando si conosce in anticipo quante iterazioni devono essere eseguite.

### Sintassi

```
for (inizializzazione; condizione; incremento) {  
    // istruzioni  
}
```

### Esempio pratico

```
for (int i = 0; i < 5; i++) {  
    IO.println("Iterazione: " + i);  
} // stampa da 0 a 4
```

### Confronto con while

|                  | for                       | while                           |
|------------------|---------------------------|---------------------------------|
| Uso ideale       | Numero di iterazioni noto | Iterazioni basate su condizione |
| Inizializzazione | Inclusa nella sintassi    | Fuori dal ciclo                 |
| Leggibilità      | Più concisa per contatori | Meno compatta per cicli contati |

Esempio equivalente al precedente con while:

```
int i = 0;  
while (i < 5) {  
    IO.println("Iterazione: " + i);  
    i++;  
}
```

### Errori comuni

- Ciclo infinito, dimenticando di aggiornare la variabile di controllo:

```
for (int i = 0; i < 5; /* manca i++ */) {  
    IO.println(i);  
}
```

- Dimenticarsi di inserire il punto e virgola tra le condizioni nelle parentesi:

```
for (int i = 10; i < 5 i++) { // Errore  
    // ...  
} // stesso se si usa la virgola
```

- Punto e virgola dopo la parentesi:

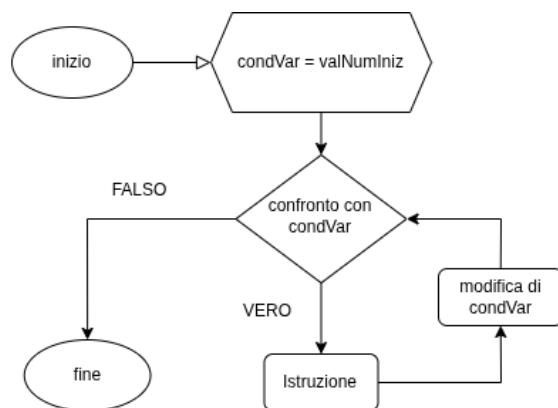
```
for (int i = 0; i < 5; i++); { // ; termina il ciclo  
    IO.println(i); // Eseguito una sola volta  
}
```

- Uso di variabili dichiarate nel ciclo for e pertanto non accessibili fuori:

```
for (int i = 0; i < 5; i++) {  
    // ...  
}  
IO.println(i); // Errore: i non esiste qui
```

- Condizione mai soddisfatta:

```
for (int i = 10; i < 5; i++) { // Mai vera  
    // ...  
}
```



### Nota bene

Guardando lo schema, si potrebbe dire che i costrutti while e for coincidano.

Ma la differenza sta proprio nella modifica della variabile di controllo.

Mentre il costrutto while permette la modifica in qualsiasi punto all'interno dell'insieme di istruzioni, nel costrutto for avviene solo dopo aver eseguito tutte le istruzioni (cioè in uscita).

## for - each

Il costrutto for-each in Java (noto anche come enhanced for loop) permette di scorrere elementi di un array o di una collezione in modo semplice e sicuro, senza dover gestire indici o iteratori espliciti.

### Sintassi

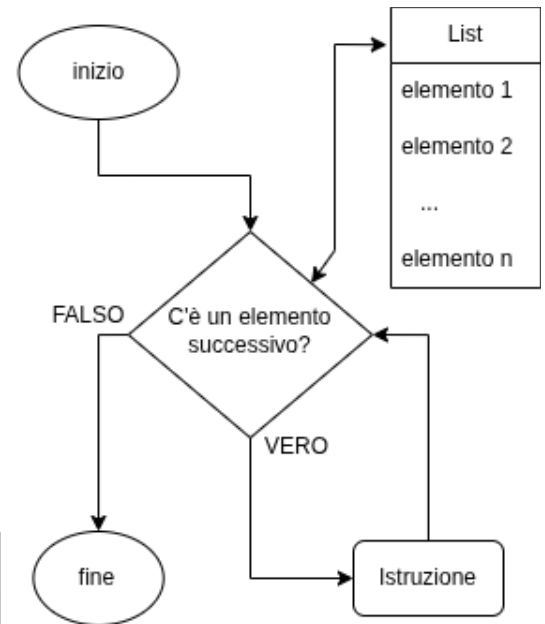
```
for (type elemento : collezione) {  
    // istruzioni  
}
```

### Esempio pratico

```
String [] nomi = {"Anna", "Luca", "Maria"};  
for (String nome : nomi) {  
    IO.println(nome);  
} //Stampa tutti i nomi nell'array
```

### Confronto con for tradizionale

|                     | for – each    | for tradizionale |
|---------------------|---------------|------------------|
| Leggibilità         | Alta          | Media            |
| Gestione Indici     | Automatica    | Manuale          |
| Uso con Indici      | NON possibile | Possibile        |
| Ordinamento Inverso | NO            | SI               |
| Modifica Elementi   | Limitato      | Completo         |



### Osservazione

Sebbene questo tipo di costrutto nasca per interagire con gli elementi della lista, nulla toglie che si possa non usare alcun elemento nel blocco istruzioni, usando la lunghezza stessa della lista come contatore.

### Casi d'errore comuni

- Non si può modificare l'elemento dell'array

```
int [] numeri = {1, 2, 3};  
for (int num : numeri) {  
    num = 10; // Non modifica l'array originale  
} //La variabile num è una copia del valore
```

- Non si può accedere all'indice

```
for (String nome : nomi) {  
    // Non posso sapere la posizione di nome  
} //Se serve l'indice, usare for tradizionale
```

- Non si può iterare all'indietro

```
// Impossibile con for-each  
for (int i = array.length-1; i >= 0; i--) { ... }
```

- Non si può modificare la collezione durante l'iterazione

```
List<String> list = new ArrayList<>(Arrays.asList("A", "B"));  
for (String s : list) {  
    list.remove(s); // ConcurrentModificationException  
}
```

La sintassi <> indica i generics, introdotti in Java 5, permettono di creare classi, interfacce e metodi che operano su tipi parametrizzati, garantendo sicurezza a tempo di compilazione.

<String> = "questo oggetto usa solo Stringhe"

Java usa un "contatore invisibile" per scorrere la lista.

Se rimuovi un elemento, la lista cambia, ma il contatore non lo sa e restituisce errore!

In generale (questo consiglio vale per ogni costrutto iterativo) è bene non modificare liste di elementi che fanno parte della condizione di iterazione.